

End-to-End CI/CD Pipeline on Kubernetes

Jenkins • Docker • Kubernetes • Helm • Argo CD • Terraform

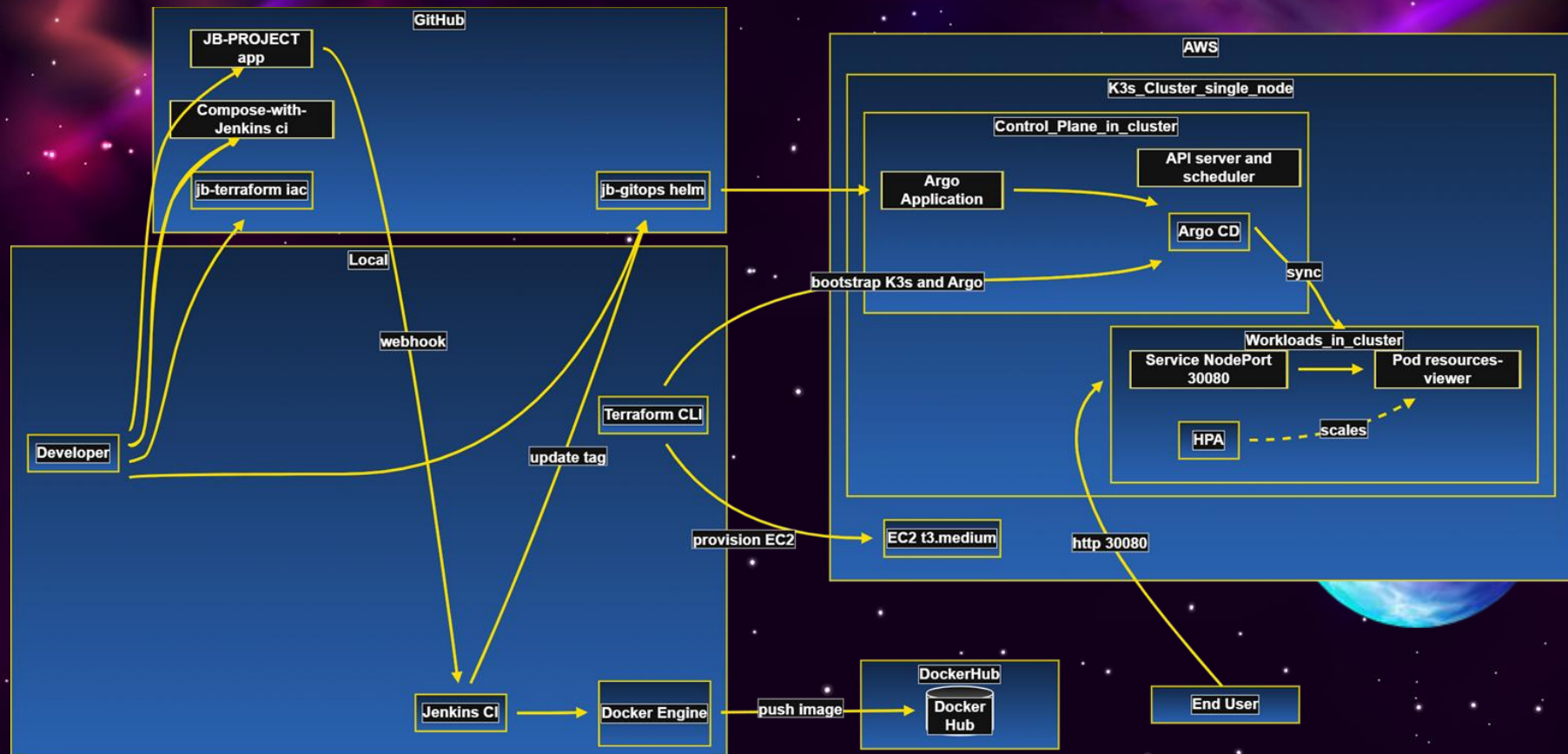
Final Project – DevOps Program (John Bryce)

Presented by: Alexander Yasheyev

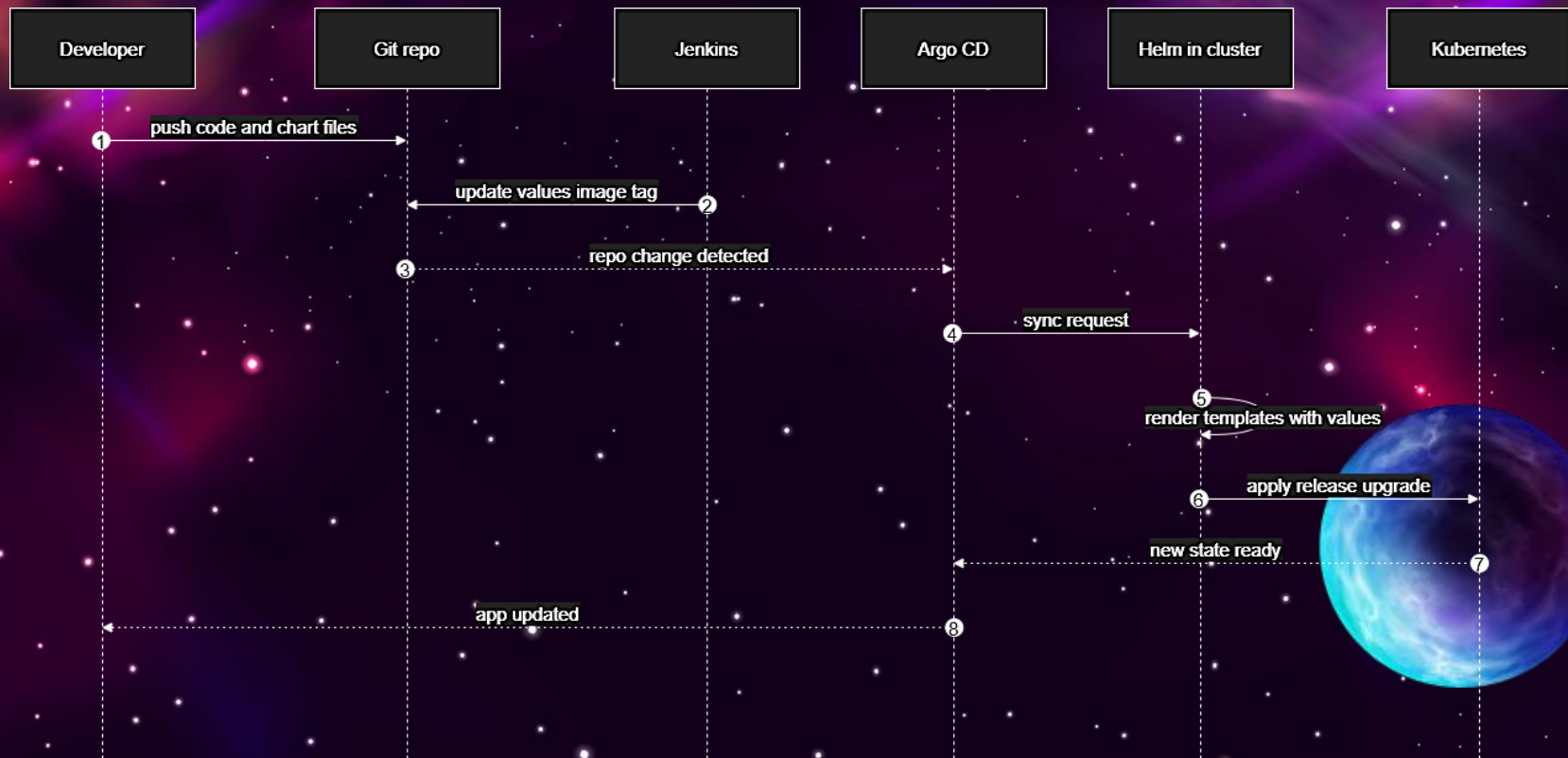
02/10/2025



*High-level system architecture demonstrating a single-node K3s cluster,
local Terraform provisioning, multi-repo CI/CD and GitOps-driven deployment flow from developer commit to end-user delivery.*



Main Flow



Multi-Repository Structure Overview

Each repository plays a unique role in the end-to-end CI/CD pipeline — from infrastructure provisioning to automated deployment via GitOps.

Repository for Continuous Integration Infrastructure

Docker Compose setup running Jenkins master and agent for building, testing, and pushing Docker images to Docker Hub.

GitHub Repository - <https://github.com/githuber20202/Compose-with-Jenkins-git>

Compose-with-Jenkins-/

- ├─ docker-compose.yml
- ├─ Dockerfile.jenkins
- ├─ jenkinsfile
- ├─ .gitignore
- ├─ README.md
- ├─ jenkins_home/ (runtime, not tracked)
- ├─ jenkins_agent/ (runtime, not tracked)

JB-PROJECT - Application Source Code

Flask-based web application that displays AWS EC2 and related cloud resources using boto3.

GitHub Repository - <https://github.com/githuber20202/JB-PROJECT.git>

JB-PROJECT/

- ├─ app.py
- ├─ Dockerfile
- ├─ requirements.txt
- ├─ templates/
 - ├─ index.html
- ├─ static/
 - ├─ css/
 - ├─ style.css
 - ├─ images/
 - ├─ background.png
- ├─ Readme.md
- ├─ .dockerignore
- ├─ .gitignore

jb-terraform - Infrastructure as Code (IaC)

Terraform configuration for provisioning AWS infrastructure, EC2 instance, security groups, and optional K3s cluster setup.

GitHub Repository - <https://github.com/githuber20202/jb-terraform.git>

jb-terraform/

- ├─ main.tf
- ├─ variables.tf
- ├─ outputs.tf
- ├─ provider.tf
- ├─ user-data.sh
- ├─ terraform.tfvars (local, gitignored)
- ├─ .gitignore
- ├─ README.md

jb-gitops - Kubernetes Deployment via Helm

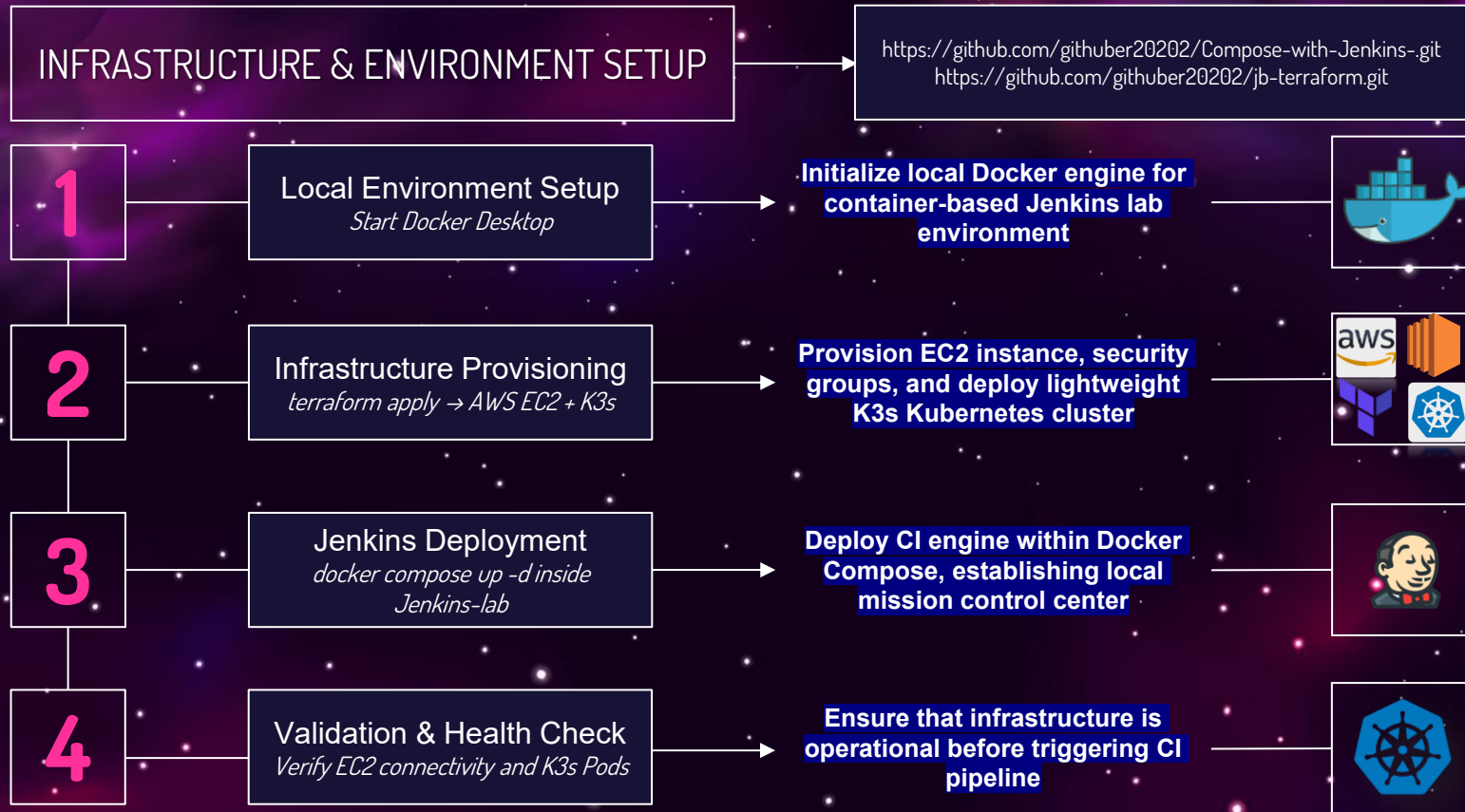
Helm chart repository responsible for deploying the application to Kubernetes and managing synchronization with Argo CD.

GitHub Repository - GitOps Repo - <https://github.com/githuber20202/jb-gitops.git>

jb-gitops/

- ├─ aws-resources-viewer/
 - ├─ Chart.yaml
 - ├─ values.yaml
 - ├─ .helmignore
 - ├─ templates/
 - ├─ deployment.yaml
 - ├─ service.yaml
 - ├─ serviceaccount.yaml
 - ├─ hpa.yaml
 - ├─ rbac.yaml
 - ├─ _helpers.tpl

Phase 1 – Infrastructure Launchpad / Pre-flight Setup

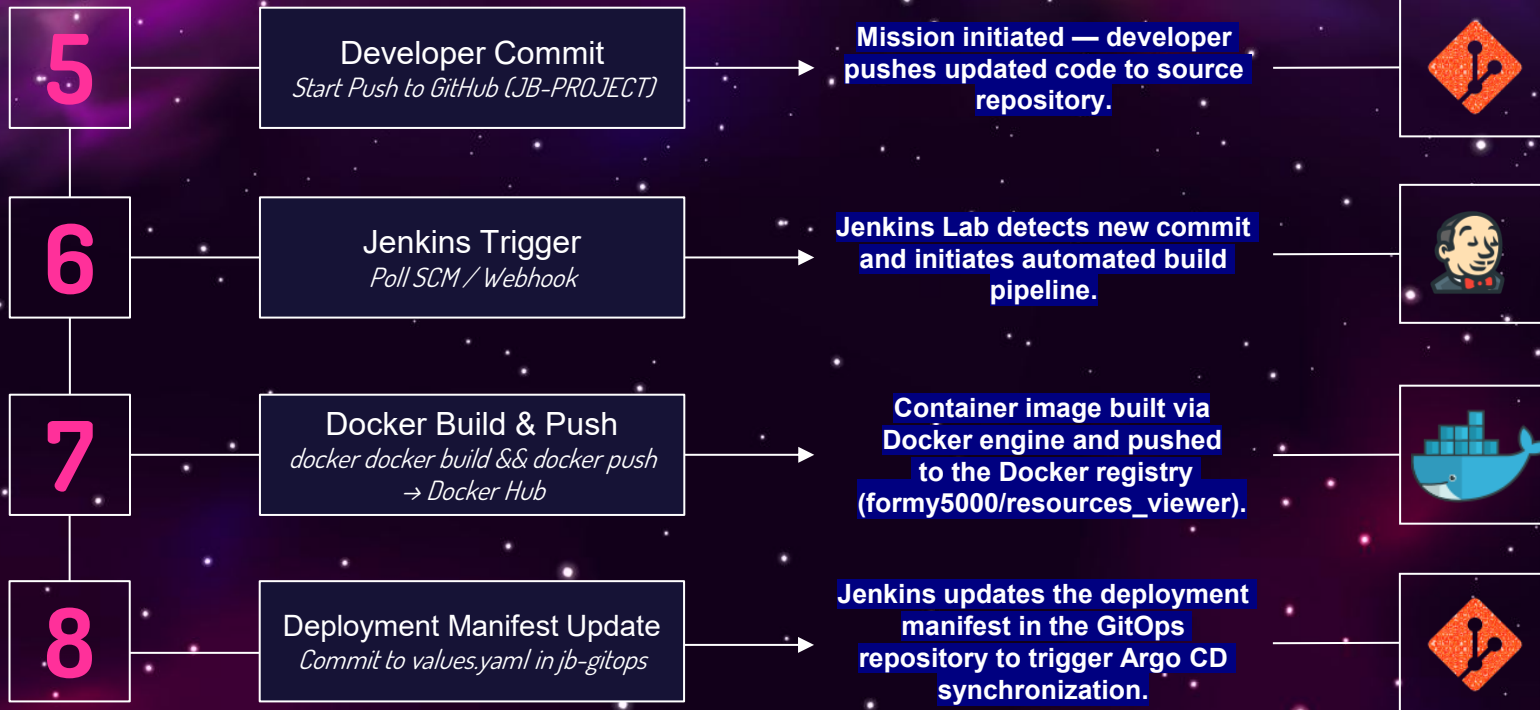


"Provisioning complete – ready for CI liftoff."

Phase 2 – Continuous Integration / Mission Control Operations

“Automating build and artifact delivery through Jenkins Lab pipelines.”

CI Pipeline - <https://github.com/githuber20202/Compose-with-Jenkins-.git>
Application Source - <https://github.com/githuber20202/JB-PROJECT.git>
Docker Registry - [formy5000/resources_viewer](https://formy5000.com/resources_viewer)
GitOps Repo - <https://github.com/githuber20202/jb-gitops.git>

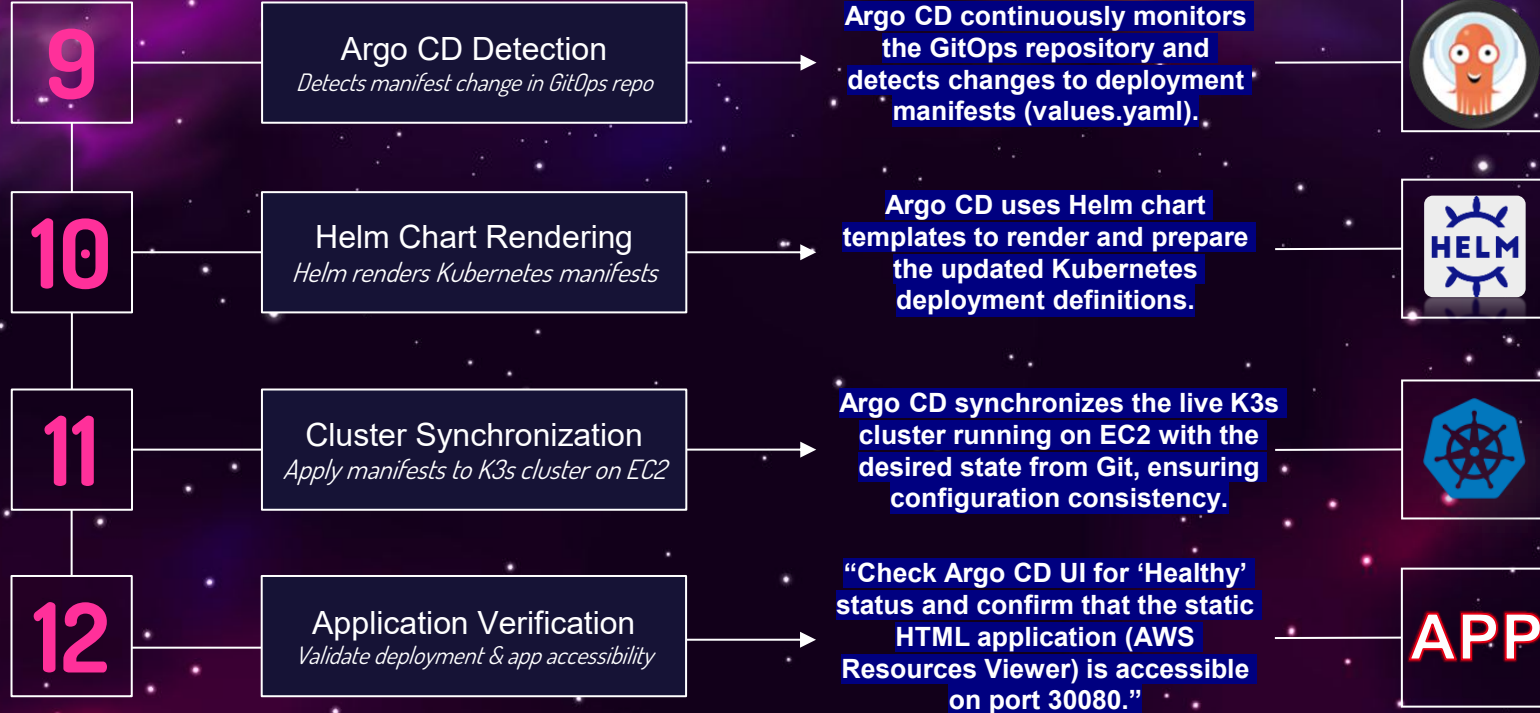


“Integration complete — image successfully deployed to orbit awaiting GitOps sync.”

Phase 3 – Continuous Deployment / GitOps Autopilot

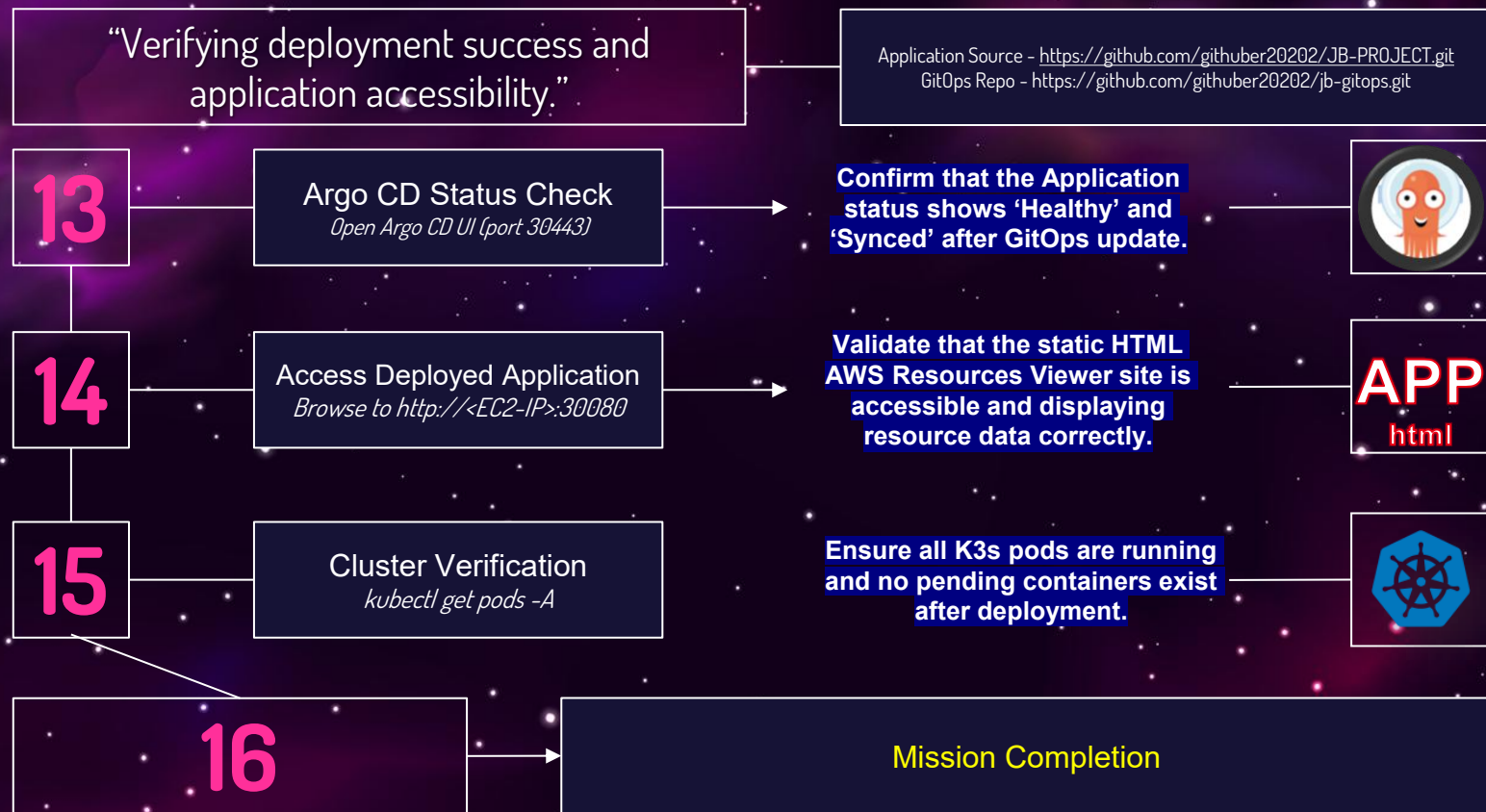
“Automating Kubernetes deployments through Helm and Argo CD synchronization.”

Application Source - <https://github.com/githuber20202/JB-PROJECT.git>
GitOps Repo - <https://github.com/githuber20202/jb-gitops.git>



“Deployment complete — Argo CD synchronized and Flask application live on EC2 orbit.”

Phase 4 – Mission Accomplished / Application Orbit



“Mission accomplished — the AWS Resources Viewer is live on EC2, fully deployed through the CI/CD + GitOps pipeline.”